Exercícios de C

Lista de prática organizada por tópico, do básico aos mini projetos. Pensada para estudar em ANSI C (C89).

Como usar

116 exercícios em 12 tópicos, seguindo a ordem em que o conteúdo costuma ser ensinado. Cada exercício traz o nível entre colchetes: **[F]** fácil, **[M]** médio, **[D]** difícil. Marque o quadrado quando terminar.

Compile sempre com avisos ligados e corrija todos eles:

```
gcc -ansi -pedantic -Wall -Wextra prog.c -o prog -lm
gcc -g -fsanitize=address prog.c -o prog
```

Dicas de estudo: tente resolver sem olhar a resposta, por pelo menos 20 minutos; rode o programa com entradas estranhas (zero, negativo, vazio, muito grande); e quando algo travar, use o `gdb` em vez de adivinhar. Em cada tópico, faça primeiro os [F], depois os [M]; os [D] podem ficar para uma segunda passada.

Observação sobre a origem: as listas que encontrei na web (de universidades e institutos, além do Exercism) têm direitos autorais, então não copiei os enunciados. Os exercícios deste PDF são autorais, escritos para cobrir os mesmos tópicos que essas listas costumam cobrir. A última página traz os links para você buscar mais material.

1. Primeiros passos: variáveis, operadores e E/S

Foco: tipos, `printf`, `scanf` e operadores aritméticos.

[ ] 01 [F] Escreva um programa que imprima seu nome e sua cidade em duas linhas.

[ ] 02 [F] Leia a base e a altura de um retângulo (`double`) e mostre a área e o perímetro com 2 casas decimais.

[ ] 03 [F] Leia uma temperatura em Celsius e converta para Fahrenheit e Kelvin.

[ ] 04 [F] Leia dois inteiros e mostre soma, diferença, produto, quociente inteiro e resto da divisão.

[ ] 05 [F] Leia um valor em reais e a cotação do dólar. Mostre o valor convertido.

[ ] 06 [M] Leia um número de segundos (ex.: 3725) e mostre quantas horas, minutos e segundos isso representa (1h 2min 5s).

[ ] 07 [M] Leia três notas com pesos 2, 3 e 5 e calcule a média ponderada.

[ ] 08 [M] Troque o valor de duas variáveis inteiras **sem** usar variável auxiliar (dica: aritmética ou XOR).

[ ] 09 [M] Leia um número de 3 dígitos e imprima seus dígitos separados (ex.: 472 vira 4 7 2), usando só `/` e `%`.

[ ] 10 [D] Mostre o tamanho em bytes de cada tipo básico na sua máquina com `sizeof`, e os limites de `int` e `char` via `limits.h`.

2. Condicionais: if, else e switch

Foco: decisões, operadores lógicos e `switch`.

[ ] 11 [F] Leia um inteiro e diga se é par ou ímpar.

[ ] 12 [F] Leia dois números e mostre o maior. Depois faça o mesmo com três números.

[ ] 13 [F] Leia a idade de uma pessoa e classifique: criança (até 11), adolescente (12 a 17), adulto (18 a 59) ou idoso (60+).

[ ] 14 [F] Leia um número e informe se é positivo, negativo ou zero.

[ ] 15 [M] Leia três lados e diga se formam um triângulo. Se formarem, classifique em equilátero, isósceles ou escaleno.

[ ] 16 [M] Leia um ano e diga se é bissexto (divisível por 4, exceto séculos, que precisam ser divisíveis por 400).

[ ] 17 [M] Resolva uma equação do 2º grau $ax^2 + bx + c = 0$, tratando os casos de delta negativo, zero e positivo (use `math.h`).

[ ] 18 [M] Crie uma calculadora: leia dois números e um operador (`+ - * /`) e use `switch`. Evite divisão por zero.

[ ] 19 [M] Leia um número de 1 a 7 e mostre o dia da semana correspondente com `switch`.

[ ] 20 [D] Leia uma data (dia, mês, ano) e valide se ela existe, considerando meses de 28 a 31 dias e anos bissextos.

3. Laços: for, while e do-while

Foco: repetição, acumuladores, contadores e laços aninhados.

[ ] 21 [F] Imprima os números de 1 a 100.

[ ] 22 [F] Leia um número N e imprima a tabuada dele de 1 a 10.

[ ] 23 [F] Calcule a soma dos N primeiros números naturais.

[ ] 24 [F] Leia números até o usuário digitar 0 e mostre a soma e a quantidade lida.

[ ] 25 [M] Calcule o fatorial de N com um laço (sem recursão).

[ ] 26 [M] Imprima os N primeiros termos da sequência de Fibonacci.

[ ] 27 [M] Diga se um número é primo. Depois imprima todos os primos até N.

[ ] 28 [M] Leia N números e mostre o maior, o menor e a média.

[ ] 29 [M] Imprima este padrão para um N lido (exemplo com N = 4):

```
*
**
***
****
```

[ ] 30 [M] Inverta um número inteiro (ex.: 1234 vira 4321) e verifique se ele é palíndromo.

[ ] 31 [D] Calcule o MDC de dois números pelo algoritmo de Euclides e o MMC a partir dele.

[ ] 32 [D] Converta um número decimal para binário usando divisões sucessivas.

[ ] 33 [D] Jogo de adivinhação: o programa sorteia um número de 1 a 100 (`rand`, `srand`, `time`) e dá dicas de maior/menor até você acertar. Mostre as tentativas.

4. Funções e recursão

Foco: protótipos, passagem por valor, escopo e recursividade.

[ ] 34 [F] Crie a função `int maior(int a, int b)` e use-a no `main`.

[ ] 35 [F] Crie `int ehPar(int n)` que devolva 1 ou 0.

[ ] 36 [M] Crie `void troca(int *a, int *b)` e teste. Explique por que sem ponteiros não funciona.

[ ] 37 [M] Escreva `long fatorial(int n)` de forma recursiva.

[ ] 38 [M] Escreva `int fib(int n)` recursivo e compare o tempo com a versão em laço para n = 40.

[ ] 39 [M] Crie `double potencia(double base, int exp)` sem usar `pow`, tratando expoente negativo.

[ ] 40 [M] Crie `int somaDigitos(int n)` de forma recursiva.

[ ] 41 [D] Implemente a Torre de Hanói recursiva, imprimindo cada movimento para N discos.

[ ] 42 [D] Escreva `int mdc(int a, int b)` recursivo.

[ ] 43 [D] Crie uma função que calcule uma raiz quadrada aproximada pelo método de Newton, parando quando a diferença for menor que 1e-6.

5. Vetores (arrays unidimensionais)

Foco: percorrer, buscar, ordenar e passar vetores para funções (lembre de enviar o tamanho).

[ ] 44 [F] Leia 10 inteiros num vetor e imprima-os na ordem inversa.

[ ] 45 [F] Calcule a soma e a média dos elementos de um vetor.

[ ] 46 [F] Encontre o maior elemento e a posição dele.

[ ] 47 [M] Conte quantos elementos de um vetor são pares e quantos são ímpares.

[ ] 48 [M] Verifique se um valor lido existe no vetor (busca linear) e mostre a posição.

[ ] 49 [M] Inverta o vetor **no próprio vetor**, trocando extremos, sem criar outro.

[ ] 50 [M] Remova os elementos repetidos de um vetor.

[ ] 51 [M] Ordene um vetor com o Bubble Sort. Depois implemente Selection Sort e compare.

[ ] 52 [D] Dado um vetor ordenado, implemente a busca binária, de forma iterativa e depois recursiva.

[ ] 53 [D] Intercale (merge) dois vetores já ordenados num terceiro vetor ordenado.

[ ] 54 [D] Gire os elementos de um vetor K posições para a direita, sem usar vetor auxiliar grande.

6. Matrizes

Foco: laços aninhados e índices `[linha][coluna]`.

[ ] 55 [F] Leia uma matriz 3x3 e imprima-a formatada.

[ ] 56 [F] Some todos os elementos de uma matriz.

[ ] 57 [M] Mostre a soma de cada linha e de cada coluna.

[ ] 58 [M] Calcule a soma da diagonal principal e da diagonal secundária de uma matriz quadrada.

[ ] 59 [M] Gere a transposta de uma matriz MxN.

[ ] 60 [M] Some e subtraia duas matrizes de mesmas dimensões.

[ ] 61 [D] Multiplique duas matrizes (verifique se as dimensões são compatíveis).

[ ] 62 [D] Verifique se uma matriz quadrada é simétrica.

[ ] 63 [D] Crie um jogo da velha 3x3 para dois jogadores, detectando vitória e empate.

[ ] 64 [D] Imprima a matriz em espiral (do canto superior esquerdo para o centro).

7. Strings

Foco: `char[]`, terminador `'\0'`, `string.h` e `ctype.h`. Use `fgets` para ler linhas.

[ ] 65 [F] Leia uma palavra e mostre seu tamanho **sem** usar `strlen`.

[ ] 66 [F] Conte vogais e consoantes de uma frase.

[ ] 67 [F] Converta uma string para maiúsculas e depois para minúsculas.

[ ] 68 [M] Implemente suas próprias versões de `strcpy`, `strcat` e `strcmp`.

[ ] 69 [M] Inverta uma string e verifique se ela é palíndroma (ignorando espaços e maiúsculas).

[ ] 70 [M] Conte quantas palavras há numa frase.

[ ] 71 [M] Substitua todas as ocorrências de um caractere por outro numa string.

[ ] 72 [M] Implemente a cifra de César com deslocamento K (cifrar e decifrar).

[ ] 73 [D] Verifique se duas palavras são anagramas.

[ ] 74 [D] Leia uma frase e imprima a palavra mais longa.

[ ] 75 [D] Escreva sua própria `atoi`, convertendo uma string numérica para inteiro, tratando sinal e caracteres inválidos.

8. Ponteiros

Foco: endereços, desreferência, aritmética de ponteiros e relação com arrays. Use o depurador (gdb) para ver os endereços.

[ ] 76 [F] Declare um inteiro e um ponteiro para ele. Imprima o valor e o endereço pelas duas vias, e altere o valor via ponteiro.

[ ] 77 [F] Imprima os endereços dos elementos de um vetor de `int` e observe a diferença entre eles.

[ ] 78 [M] Percorra um vetor usando apenas aritmética de ponteiros (`*(p + i)` e `p++`).

[ ] 79 [M] Escreva `void minMax(int *v, int n, int *min, int *max)`, que devolve dois resultados via ponteiros.

[ ] 80 [M] Reescreva `strlen` e `strcpy` usando só ponteiros, sem índices.

[ ] 81 [M] Escreva `void inverte(int *v, int n)` com dois ponteiros (início e fim) andando em direção um ao outro.

[ ] 82 [D] Use um ponteiro para função para criar `aplica(int *v, int n, int (*f)(int))`, que aplica uma função a cada elemento.

[ ] 83 [D] Crie um vetor de ponteiros para string e ordene-o alfabeticamente trocando apenas os ponteiros.

[ ] 84 [D] Use `qsort` com uma função de comparação para ordenar um vetor de inteiros em ordem decrescente.

9. struct, enum e typedef

Foco: agrupar dados, vetores de structs, ponteiro para struct e o operador `->`.

[ ] 85 [F] Crie uma struct `Aluno` (nome, matrícula, nota). Leia um aluno e imprima os dados.

[ ] 86 [M] Leia N alunos e mostre a média da turma e o aluno com maior nota.

[ ] 87 [M] Ordene um vetor de alunos por nota, e depois por nome.

[ ] 88 [M] Crie uma struct `Ponto` (x, y) e uma função que calcule a distância entre dois pontos.

[ ] 89 [M] Defina um `enum` para os dias da semana e uma função que diga se o dia é útil ou fim de semana.

[ ] 90 [M] Crie a struct `Data` e uma função que compare duas datas (anterior, igual ou posterior).

[ ] 91 [D] Faça uma agenda de contatos em memória (nome, telefone) com inserir, listar, buscar e remover.

[ ] 92 [D] Defina uma struct `Fracao` e implemente soma, subtração, produto e simplificação de frações.

[ ] 93 [D] Use uma `union` para ver os bytes de um `float` e descobrir a endianness da sua máquina.

10. Memória dinâmica

Foco: `malloc`, `calloc`, `realloc`, `free`. Teste tudo com valgrind ou `-fsanitize=address`.

[ ] 94 [F] Leia N e aloque dinamicamente um vetor de N inteiros. Preencha, mostre e libere.

[ ] 95 [M] Crie um vetor dinâmico que cresce com `realloc` conforme o usuário digita números, até digitar -1.

[ ] 96 [M] Aloque uma matriz MxN dinamicamente (vetor de ponteiros) e libere corretamente.

[ ] 97 [M] Escreva `char *copia(const char *s)`, que duplique uma string na heap.

[ ] 98 [M] Leia uma linha de tamanho desconhecido, crescendo o buffer à medida que for necessário.

[ ] 99 [D] Implemente uma lista encadeada simples com inserir no início, inserir no fim, buscar, remover e liberar tudo.

[ ] 100 [D] Implemente uma pilha e use-a para verificar se os parênteses de uma expressão estão balanceados.

[ ] 101 [D] Implemente uma fila (circular ou encadeada).

[ ] 102 [D] Implemente uma árvore binária de busca com inserção, busca e percurso em ordem.

11. Arquivos

Foco: `fopen`, `fclose`, leitura e escrita de texto e binário. Sempre verifique se `fopen` devolveu `NULL`.

[ ] 103 [F] Escreva um programa que grave 10 números num arquivo texto, um por linha.

[ ] 104 [F] Leia o arquivo anterior e mostre a soma dos valores.

[ ] 105 [M] Conte linhas, palavras e caracteres de um arquivo (um mini `wc`).

[ ] 106 [M] Copie o conteúdo de um arquivo para outro (um mini `cp`), via argumentos `argc`/`argv`.

[ ] 107 [M] Grave um vetor de structs `Aluno` num arquivo binário com `fwrite` e leia de volta com `fread`.

[ ] 108 [D] Transforme sua agenda de contatos para que ela persista em arquivo ao fechar e carregue ao abrir.

[ ] 109 [D] Leia um arquivo CSV simples (nome;nota) e mostre as estatísticas: média, maior, menor.

[ ] 110 [D] Use `fseek` e `ftell` para descobrir o tamanho de um arquivo e ler o registro de número K sem ler os anteriores.

12. Mini projetos (juntando tudo)

Escolha 1 ou 2 e faça em vários arquivos `.c`/`.h`, com Makefile.

[ ] 111 [M] Jogo da forca com lista de palavras em arquivo.

[ ] 112 [M] Cadastro de produtos com estoque, preço, busca e relatório, salvo em arquivo binário.

[ ] 113 [D] Biblioteca de matemática própria (`meumath.h`/`.c`) com testes usando `assert`.

[ ] 114 [D] Interpretador de uma calculadora de expressões (`2 + 3 * (4 - 1)`) com pilha.

[ ] 115 [D] Jogo da vida de Conway numa matriz, exibindo as gerações no terminal.

[ ] 116 [D] Um mini shell que leia comandos e imprima o que digitou, com histórico em lista encadeada.

Onde achar mais exercícios

Exercism, trilha de C
https://exercism.org/tracks/c
Mais de 80 exercícios gratuitos, com testes automáticos e mentoria. Assume que você já conhece a sintaxe básica.

Lista de exercícios de C (Fatec) no GitHub
https://github.com/diego486/fatec_linguagemC
Repositório com listas sobre repetição, vetores, matrizes, strings, structs e funções.

Artigo 'C language parte I' (Medium)
https://medium.com/@pedro.vaf/c-language-parte-i-63df035809de
Exercícios iniciais comentados; cita uma lista mais completa do IFBA.

30 exercícios básicos de C (Placement Preparation)
https://placementpreparation.io/programming-exercises/c-programming/beginners
Lista em inglês com soluções, boa para treinar leitura técnica.

Apostila de C, núcleo de acesso aberto (CORE)
https://core.ac.uk/download/161369928.pdf
Livro em PDF com exercícios de aula ao fim de cada capítulo (licença CC BY-NC-ND).

Esses links foram encontrados em buscas e não foram todos verificados por dentro; confira se continuam no ar.